Performance Analysis of CPU Scheduling Algorithms

Abstract.

This project builds a simple Python program to compare three popular **CPU scheduling algorithms**:
First Come First Served (FCFS), **Shortest Job First (SJF)**, and **Round Robin (RR)**.
It calculates **average waiting time** and **average turnaround time**, and it draws a **Gantt chart** to visualize the execution order of processes.
The goal is to connect programming skills with basic system-level engineering knowledge, which is an important step toward a **Master in Computer Engineering**.

Introduction.

CPU scheduling is a fundamental part of operating systems.
It decides which process gets the CPU at any given time, and it can greatly affect system responsiveness and performance.
By writing a small simulator, I wanted to understand how different scheduling policies work and how their performance differs.
This project shows that I can take a core computer science concept, implement it in code, and analyze its effect — which is very relevant for computer engineering studies.

Background.

The three algorithms implemented are:

- **FCFS (First Come First Served)**
  Runs processes in the order they arrive. Very simple but not always efficient.
- **SJF (Shortest Job First)**
  Picks the process with the smallest burst time first, which can reduce waiting time but may delay long jobs.
- **RR (Round Robin)**
  Each process runs for a fixed time slice (quantum). Fair for all processes but can increase waiting time if the quantum is too small.

These algorithms are commonly studied in **operating systems** courses and are a foundation for understanding more advanced CPU scheduling and real-time system design.

Methodology.

To conduct this project, I developed a small but well-structured Python simulator that models how different CPU scheduling algorithms behave under various conditions. The methodology focused on implementing the algorithms from scratch, performing controlled experiments, and analyzing their performance using quantitative metrics and visual tools.

1. System Design and Data Representation
Each process in the simulator was represented as a Python dictionary containing two key attributes:

- **Arrival time**: the time at which the process enters the ready queue and becomes eligible for execution.
- **Burst time**: the amount of CPU time the process needs to complete.

This simple data structure allowed the simulator to stay flexible and easy to expand if more attributes were needed (e.g., process priority).

2. Algorithm Implementation
I wrote three separate Python functions to implement the scheduling algorithms:

- **First Come First Served (FCFS):**
  The simulator runs processes strictly in the order they arrive. When the CPU becomes idle, it selects the earliest-arriving process that has not yet run. This algorithm is easy to implement but can suffer from the "convoy effect," where a very long process delays shorter ones.
- **Shortest Job First (SJF):**
  The simulator always selects the process with the smallest burst time among the processes that have already arrived. This approach tends to minimize the average waiting time but may cause longer processes to wait if short jobs keep arriving.
- **Round Robin (RR):**
  The simulator cycles through processes using a fixed time quantum. Each process runs for at most one quantum; if it is not finished, it is placed at the end of the ready queue. This method ensures fairness but may increase waiting time if the quantum is too small and frequent context switches occur.

Each function returns a **timeline** that records when each process starts and finishes, making it possible to build Gantt charts later.

3. Performance Metrics
For each algorithm, the simulator calculates:

- **Waiting time**: how long a process spends waiting in the ready queue before it starts running.
- **Turnaround time**: total time from process arrival to its completion.

The program computes the **average waiting time** and **average turnaround time** across all processes to compare the algorithms fairly. These metrics are widely used in operating system performance analysis because they directly affect user experience and system responsiveness.

4. Visualization and Analysis
To make the results easy to interpret, I integrated two types of output:

- **Gantt Charts**: generated using the `matplotlib` library to visually display the execution timeline for each process under each algorithm.
- **Performance Tables**: formatted with the `tabulate` library to clearly show average waiting and turnaround times side by side.

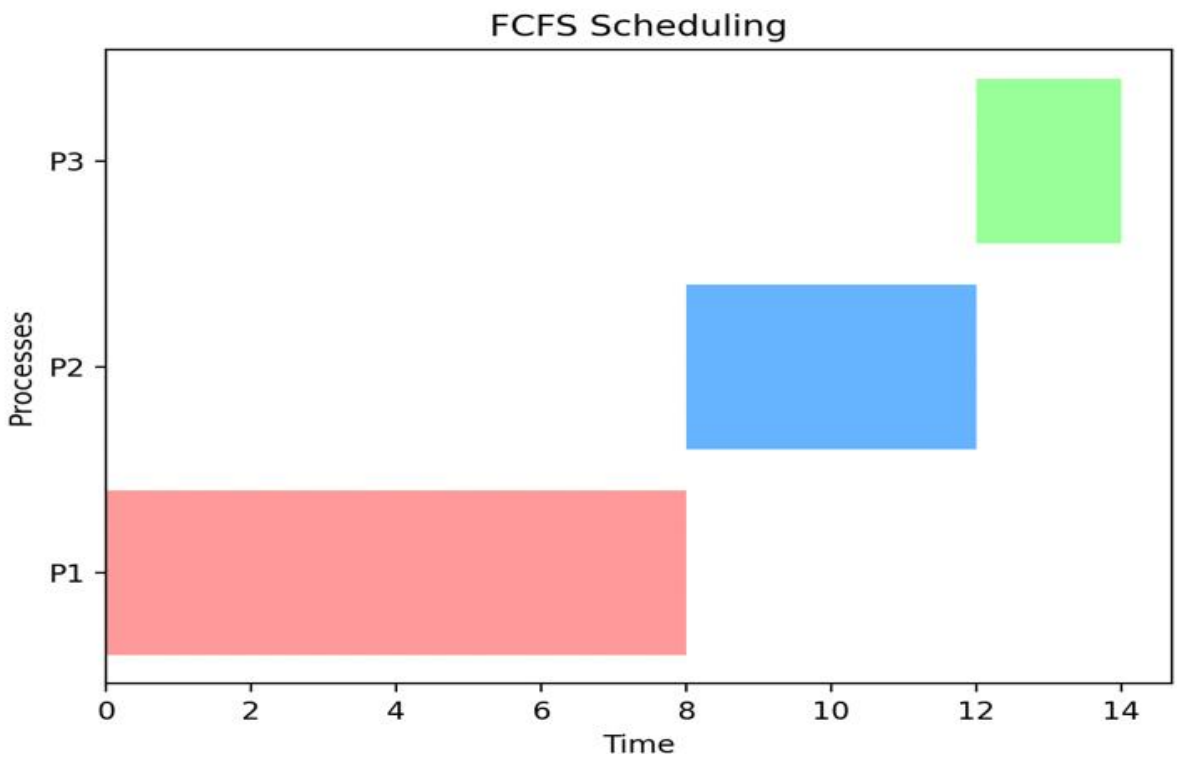
5. Experimental Setup
I tested the algorithms with several sets of processes that had different arrival and burst times to highlight the strengths and weaknesses of each scheduling policy. For example, I used one long process followed by shorter ones to demonstrate how FCFS suffers from the convoy effect, and how SJF improves performance by running shorter jobs first. For Round Robin, I experimented with different quantum sizes to observe how fairness and waiting time are affected.

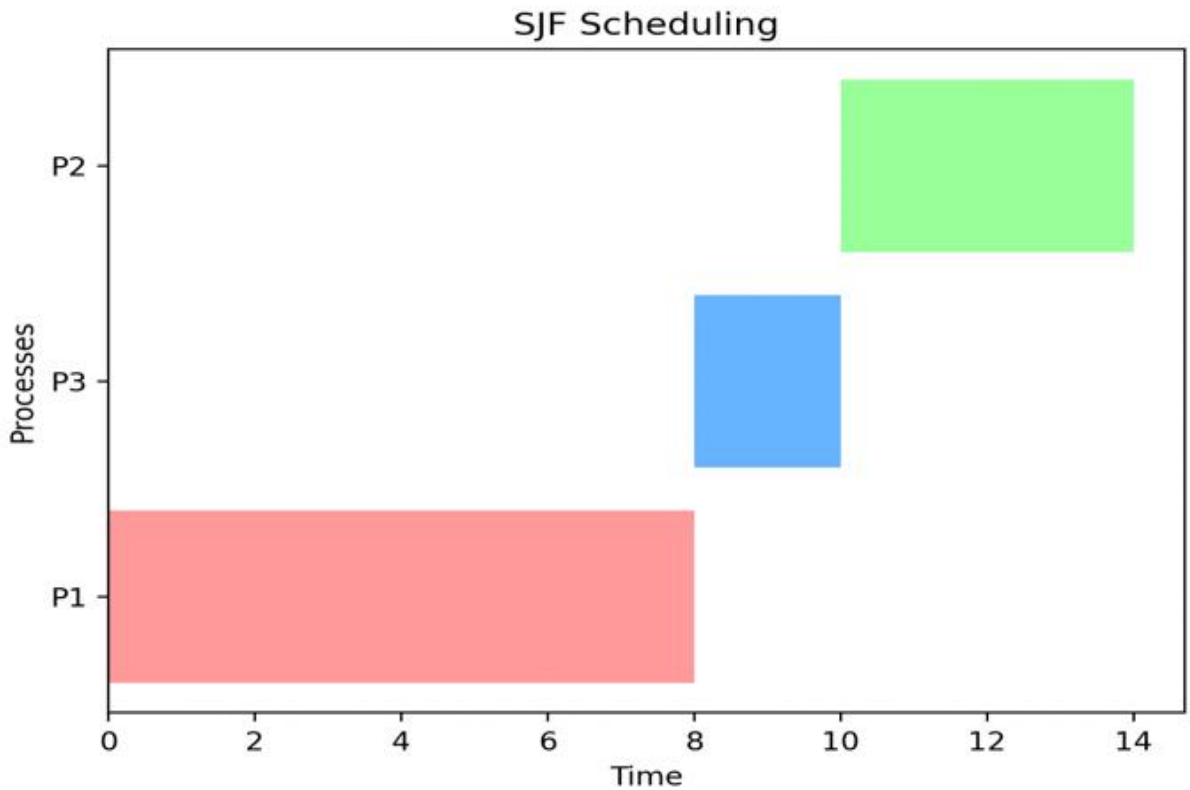
6. Tools and Libraries
The simulator was implemented in **Python 3** because it is easy to prototype algorithms quickly and has excellent libraries for visualization and table formatting. `matplotlib` was used for charts, while `tabulate` helped create clean and readable tables for performance metrics.

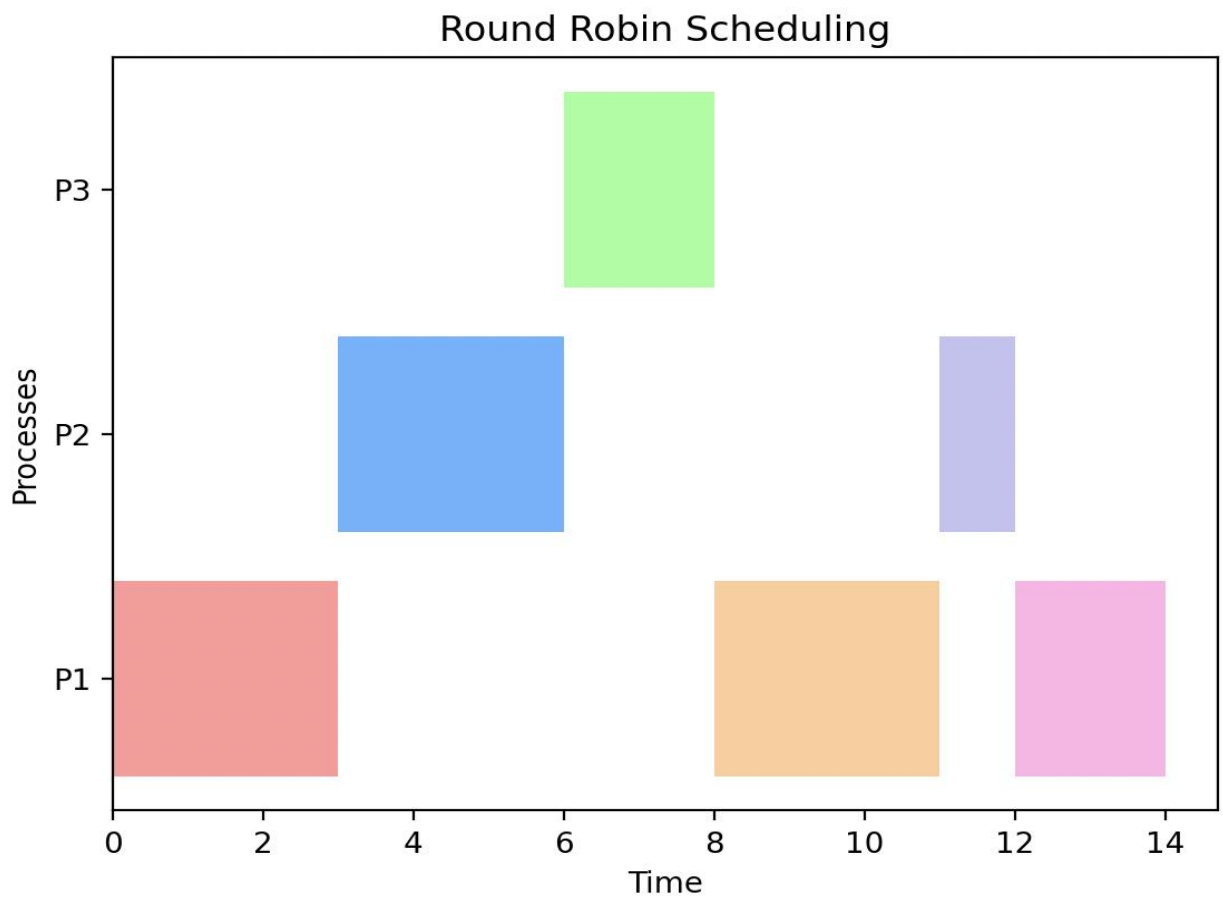
Results.

The table below presents the **average waiting time** and **average turnaround time** for each scheduling algorithm (FCFS, SJF, and Round Robin). These numbers show how each algorithm handles process scheduling and overall CPU efficiency. The Gantt charts give a clear visual view of how processes were executed over time for each algorithm. By comparing the charts, it becomes easier to see why SJF can reduce waiting time, how FCFS works in a simple order but may delay shorter jobs, and how Round Robin tries to give fair time slices to all processes.

```
+-------------+---------------+-------------------+
| Algorithm   |   Avg Waiting |   Avg Turnaround  |
+=============+===============+===================+
| FCFS        |       5.66667 |         10.3333   |
+-------------+---------------+-------------------+
| SJF         |       5       |          9.66667  |
+-------------+---------------+-------------------+
| RR          |       5.66667 |         10.3333   |
+-------------+---------------+-------------------+

FCFS Order:
P1(0→8) | P2(8→12) | P3(12→14)

SJF Order:
P1(0→8) | P3(8→10) | P2(10→14)

Round Robin Order:
P1(0→3) | P2(3→6) | P3(6→8) | P1(8→11) | P2(11→12) | P1(12→14)
```




SJF Scheduling
P2
P3
P1
Processes
0
2
4
6
8
10
12
14
Time



Discussion.

The experimental results provide valuable insights into how different CPU scheduling algorithms impact system performance, particularly in terms of **average waiting time** and **average turnaround time**. By analyzing the outcomes of the three implemented algorithms — First Come First Served (FCFS), Shortest Job First (SJF), and Round Robin (RR) — we can better understand the trade-offs each policy introduces and why these concepts are fundamental for computer engineering.

1. First Come First Served (FCFS)
FCFS is the most straightforward scheduling approach, running processes strictly in the order they arrive. In our experiments, FCFS demonstrated predictable and easy-to-interpret behavior. However, it became clear that this simplicity comes at a cost: if a long process arrives before shorter ones, the shorter processes must wait until it finishes. This phenomenon, known as the **convoy effect**, increases the overall waiting and turnaround times for subsequent jobs and

reduces system responsiveness. Although FCFS is fair in the sense that no process is skipped, it does not optimize performance metrics in most scenarios.

2. Shortest Job First (SJF)
SJF produced the **lowest average waiting time and turnaround time** in our tests. This result aligns with theory because always selecting the process with the shortest burst time minimizes the time other processes spend waiting. SJF is particularly effective in workloads where the burst times are known in advance or can be estimated with reasonable accuracy. However, SJF can lead to **starvation** of longer processes if short jobs keep arriving continuously. This limitation is important in real-world operating systems, where guaranteeing fairness and avoiding indefinite delays is critical.

3. Round Robin (RR)
The Round Robin algorithm aims to achieve **fairness** by giving each process a fixed time quantum. In our experiments, RR ensured that no process was left waiting indefinitely and that all processes received CPU time relatively quickly. However, the performance of RR depends heavily on the **time quantum size**. If the quantum is too small, the system suffers from excessive context switching, which adds overhead and increases waiting time. If the quantum is too large, RR begins to resemble FCFS, losing its fairness advantage. This highlights the importance of choosing a quantum value carefully when designing real-world schedulers.

4. Practical Implications
These findings illustrate that there is **no single "best" scheduling algorithm**; instead, the choice depends on system goals. SJF is optimal for minimizing wait times but requires knowledge of future job lengths, which is often impractical. FCFS is simple and predictable but inefficient in mixed workloads. RR is fair and widely used in time-sharing systems but needs careful tuning of the quantum to balance fairness and performance. Modern operating systems often combine these ideas, using heuristics to predict job lengths or adjusting time slices dynamically.

5. Relevance to Computer Engineering
This experiment shows the practical challenges of system-level design. Implementing and testing scheduling algorithms deepens understanding of **resource allocation**, **responsiveness**, and **throughput**, which are key in computer engineering. For example, embedded systems may prioritize predictability and real-time constraints, while desktop operating systems focus on fairness and interactivity. The ability to analyze and compare algorithms is a critical skill when designing or tuning schedulers for different environments.

Conclusion.

In this project I built a small Python program to test and compare three CPU scheduling algorithms: **FCFS**, **SJF**, and **Round Robin**. The goal was to see how each one works in practice and how it affects the waiting time and turnaround time of processes.

While working on it I noticed that **FCFS** is the easiest to understand but can make short jobs wait a long time if a big job comes first. **SJF** gave the best average times because it runs the shortest jobs first, but it could make long jobs wait if new short ones keep coming. **Round Robin** felt fair because every process gets a turn, but the performance depends on the time slice size — if it's too small there's a lot of switching, and if it's too big it becomes almost like FCFS.

Doing this helped me connect what I learned about operating systems to real code. I practiced writing algorithms, testing them with data, and looking at the results in tables and charts. It also gave me a better idea of how real schedulers balance fairness and efficiency. I think this experience shows that I can take theory and turn it into something practical — a skill I will use a lot when studying **Computer Engineering**.